
Topolens: Visual Topology Regression via Deep Convolutional Networks

*An Empirical Investigation into CNN-Based Graph Structure Inference
from Rendered Images*

Author: Ali Ahmad 23L-2619

Track: Deep Learning Internship deliverable

Supervisor: Dr. Muhammad Aasim Qureshi

Institution: FAST NUCES Lahore

July 25, 2026

Abstract

Rendered images of graphs—diagrams, scanned figures, wireframes, biological pathway charts—are common in practice, yet the underlying graph-native representation that would let a Graph Neural Network (GNN) reason about them is frequently unavailable. Topolens investigates whether an ordinary convolutional neural network (CNN), operating purely on such rendered images, can estimate structural properties of the underlying graph—vertex count and edge count—as well as a graph-native model operating on raw topology, and, just as importantly, whether it does so through genuine structural reasoning or by exploiting incidental rendering artifacts.

To answer this, we assembled a controlled experimental pipeline spanning both synthetic and real-world data. We render 2,500 synthetic graphs, stratified across 5 generator families (Erdős–Rényi, Barabási–Albert, Watts–Strogatz, random trees, and dense Erdős–Rényi variants) and 4 node-count tiers (5–100 nodes), via Graphviz `sfdp`, and additionally hold out 1,301 real graphs from the MUTAG and PROTEINS benchmarks—including 67 PROTEINS graphs well beyond the training size ceiling—as a generalization test never seen during training or model selection. A custom 4-block CNN regressor is benchmarked against two baselines: a 3-layer GCN restricted to a minimal, degree-only node feature (isolating topology-as-geometry from topology-as-connectivity), and a non-learned density-based graph-statistic heuristic.

On the headline comparison, the CNN outperforms the GCN baseline on every metric across both splits—by 4–6 $\times$ on synthetic data and roughly 2 $\times$ on real data—answering the core competitiveness question in the affirmative for this setup. However, a substantial share of this report is devoted to interrogating *why*, since a single accuracy number cannot distinguish genuine structural learning from a rendering-specific shortcut. Using Grad-CAM, a node-size ablation probe validated with paired Wilcoxon signed-rank tests, a layout-sensitivity probe, a pixel-only baseline, a systematic failure taxonomy, and a live-app validation round, we find that the CNN’s accuracy is real but only partially attributable to topological understanding: a significant portion is driven by a node-size rendering confound, where the renderer’s node-radius formula inadvertently encodes vertex count into an exploitable visual cue. Removing that cue via a constant-node-size ablation roughly triples vertex-prediction error (Wilcoxon $p = 3.58 \times 10^{-6}$), while a pixel-only ink-coverage baseline shows the CNN is not merely counting dark pixels either, since that proxy’s explanatory power collapses on real-world data while the CNN’s own accuracy does not.

We further find that out-of-distribution graph size, rather than visual density or clutter, is the single dominant failure mode identified across every evaluation in this report, and that the GCN-vs-CNN comparison’s fairness is itself qualified by the GCN’s deliberately minimal feature budget. Both findings are presented not as incidental caveats but as substantive results in their own right, on equal footing with the headline accuracy numbers.

Contents

Abstract	i
1 Introduction & Motivation	1
2 Background & Problem Statement	2
2.1 Problem Statement	2
2.2 Motivation: Personal Context	3
3 Dataset Description	3
3.1 Synthetic Dataset Pool	3
3.2 Held-out Real-World Dataset	4
3.3 Split and Crosstabulation Statistics	5
4 Rendering Pipeline	8
5 Model Architecture	9
5.1 Custom CNN Regressor (<code>CustomCNNRegressor</code>)	10
5.2 GCN Baseline (<code>GraphCountGCN</code>)	10
5.3 Graph-Statistic Baseline	11
5.4 Log Target Transformation	11
6 Training Setup	11
7 Results	12
7.1 Baseline Comparison	12
7.2 Detailed Breakdown Reports	12
8 Model Interpretation & Failure Analysis	14
8.1 Grad-CAM Spatial Attributions	15
8.2 Shortcut-Learning Probe (Node-Size)	17
8.3 Layout-Sensitivity Probe	19
8.4 Wilcoxon Signed-Rank Tests	19
8.5 Ink-Coverage Correlational Analysis	20
8.6 Failure Taxonomy Analysis	21

8.7	Structural Attribute Correlations	23
8.8	Warning Banner Validation	23
8.9	Live-App Validation Panel	23
8.9.1	Part 1: Deployed App Spot Check	24
8.9.2	Part 2: Novel-Image Sanity Check	25
8.9.3	Part 3: Combinatorial Plausibility Violations	25
9	Discussion	25
9.1	On Competitiveness	26
9.2	On Mechanism	26
9.3	On the Fairness of the Comparison	27
10	Limitations	27
11	Future Work	29
12	Conclusion	30
13	References	30
	Appendix	31

List of Figures

3.1	Distribution of graph counts across the synthetic size tiers.	4
3.2	Overall node and edge count distributions across the dataset.	5
3.3	Node and edge distributions broken down by generator family.	6
4.1	Sample rendered images from the dataset showcasing different topologies.	9
7.1	Scatter plots of predictions vs. ground truth on synthetic test data.	13
7.2	Scatter plots of predictions vs. ground truth on real held-out data.	13
8.1	Grad-CAM activations mapped across representatives of each generator family. . . .	16
8.2	Grad-CAM activations on high-error failure cases.	17
8.3	Side-by-side comparison of original, constant node-size, and alternative layouts. . . .	18
8.4	Error comparison (MAE) across the different probe variants.	19
8.5	Correlations between visual pixel properties and actual node counts.	20
8.6	Plotting prediction error against physical graph size and structural density.	21
8.7	Grid of worst-performing outlier images yielding highest regression errors.	22

List of Tables

3.1	Train Split Sample Counts by Generator and Tier ($n = 1,750$)	7
3.2	Validation Split Sample Counts by Generator and Tier ($n = 375$)	7
3.3	Test Split Sample Counts by Generator and Tier ($n = 375$)	7
3.4	Held-out (Real-world) Split Sample Counts by Source and Tier ($n = 1,301$)	8
7.1	Baseline Comparison Metrics on Test and Held-out Splits	13
7.2	Detailed MAE by Generator Type on the Test Split	14
7.3	Detailed MAE by Dataset on the Held-out Split	14
7.4	Detailed MAE by Density Bucket on the Held-out Split	14
8.1	Probe Summary MAE by Variant and Size Tier	19
8.2	Wilcoxon Signed-Rank Test Results for Node-Size and Layout Probes ($n = 40$)	20
8.3	Pearson Correlations (r) for Low-Level Pixel Statistics vs. True and Predicted Graph Properties	20
8.4	Failure Taxonomy Error Analysis (CNN Regressor)	23
8.5	Warning Banner Size-Binned Error Validation	24
8.6	Predict Deployed App Spot Check Results ($n = 23$)	24
8.7	Predict App Novel-Image Sanity Check Results	25

1 Introduction & Motivation

Graph structures are ubiquitous in computer science, biology, and chemistry. Typically, machine learning models that analyze graph structures operate directly on graph-native formats (such as adjacency lists or node-edge features) using Graph Neural Networks (GNNs). However, in many real-world scenarios, the underlying graph-native representation is unavailable. We may only have access to rendered visualizations, such as legacy diagrams, scanned research papers, user interface wireframes, biological pathway figures, or hand-drawn schematics. In each of these cases, the graph exists only as pixels: there is no adjacency list to parse, no node-feature matrix to load, and often no way to recover one automatically. If a system is ever to reason about the topology of such an image, it must do so directly from the rendered picture, which is precisely the scenario this project sets out to probe.

Consider a concrete instance of this gap: a legacy engineering schematic scanned from a decades-old technical manual, a hand-drawn network sketch photographed off a whiteboard after a design meeting, or a molecular structure diagram embedded as a static figure in a scanned research paper with no accompanying data file. In every one of these cases, a human reader can glance at the picture and form a rough sense of how large or how densely connected the underlying structure is—roughly how many components it has, whether it looks sparse or tangled—without literally counting every node and edge by hand. The natural question this project asks is whether a machine can do something similar: not by first painstakingly reconstructing the exact graph structure from the image and then computing statistics over it, but by learning to estimate those statistics directly from the pixels, in much the same incidental way a person forms a quick visual impression of a diagram’s scale.

The Topolens project asks a deliberately contrarian question: if a graph is instead rendered as an image and handed to an ordinary Convolutional Neural Network (CNN), how much of its structure can still be recovered, and by what means?

This is not a claim that CNNs should replace GNNs. Rather, it is a controlled comparison designed to expose what a purely visual model can and cannot infer about topological structure, and—just as importantly—how it achieves its predictions. Two sub-questions follow directly from this framing and organize this work:

1. **Competitiveness:** Can a CNN trained on rendered graph images match or beat a GNN trained on the same underlying graphs, for the task of predicting vertex and edge counts, on both in-distribution synthetic data and out-of-distribution real-world data?
2. **Mechanism:** If the CNN performs well, is it doing something that resembles structural reasoning, or is it exploiting rendering artifacts (such as node size, layout geometry, or ink density) that happen to correlate with the labels but would not survive a change in rendering conventions?

Framing the project this way, as a controlled comparison rather than as a leaderboard exercise aimed purely at maximizing accuracy, matters for how the rest of this report should be read. It would have been easy to report a single headline accuracy number and stop there; the more useful contribution of a project like this lies in exposing the conditions under which that number can be trusted and the conditions under which it cannot. An image-based estimator that performs well only because

of an accidental rendering convention is not obviously useful outside the narrow setting where that convention happens to hold, and a report that failed to check for this would risk overstating what has actually been demonstrated. This is why comparable weight is given in this report to the process of building the pipeline correctly (Sections 2–7) as to interrogating the resulting model’s behavior before trusting it (Section 8), rather than treating the interpretability analysis as an afterthought once the accuracy numbers were already in hand.

By answering these questions, we demonstrate the strengths and major failure modes of image-based topology regression, qualifying the quantitative performance with a detailed analysis of shortcut learning. The remainder of this report is organized to build toward that answer step by step: Section 2 formalizes the prediction task and the evaluation criteria; Section 3 describes how the synthetic and real-world graph datasets were assembled; Section 4 documents exactly how graphs are turned into images, since the rendering choices made there turn out to matter a great deal; Section 5 and Section 6 describe the CNN, the GCN baseline, and how both were trained; Section 7 reports the headline quantitative comparison; Section 8 is where the Mechanism question is actually answered, through Grad-CAM, targeted probes, and statistical testing; and Sections 9–12 discuss what the combined evidence means, where it should not be over-trusted, and what a natural next iteration of this project would look like.

2 Background & Problem Statement

2.1 Problem Statement

The objective of this project is to solve the problem of graph topology regression from images. Specifically, given a single rendered image of a graph, the model must output two continuous values which are subsequently rounded to non-negative integers representing the predicted vertex count (V) and edge count (E).

Formally, let $G = (V, E)$ be a graph, and $I = \mathcal{R}(G)$ be its rendered image, where $\mathcal{R}$ is a rendering function mapping the topological structure to a 2D pixel space. The model $f(I)$ seeks to predict $y = [|V|, |E|]$. We benchmark this visual prediction pipeline against a graph-native Graph Convolutional Network (GCN) baseline $g(G)$ and a simple graph-statistic heuristic. A successful solution must achieve low Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) on both synthetic in-distribution test images and out-of-distribution real-world images from benchmark chemical and biological datasets.

The task is deliberately framed as regression rather than classification. Vertex and edge counts are unbounded, ordinally meaningful quantities (a graph with 40 nodes is topologically closer to one with 41 nodes than to one with 4), so collapsing them into discrete class buckets would throw away exactly the ordering information a useful estimator needs to preserve. MAE and RMSE are reported together rather than either alone because they answer different questions: MAE gives a robust, outlier-resistant sense of typical error, while RMSE’s squaring penalizes the occasional catastrophic miss more heavily—and, as the failure-taxonomy results in Section 8 show, this project does produce a long tail of large misses on out-of-distribution graphs, so relying on MAE alone would understate how bad those failures are. Evaluating on both an in-distribution synthetic test split and an out-of-distribution real-world held-out split is likewise deliberate: a model that only

works on graphs drawn from the same generator family it was trained on is of limited practical use, so the held-out MUTAG/PROTEINS split exists specifically to stress-test generalization beyond the synthetic distribution.

2.2 Motivation: Personal Context

This project was undertaken as the assigned scope of the author’s Deep Learning internship at FAST NUCES Lahore, under the supervision of Dr. Muhammad Aasim Qureshi. The specific problem—inferring graph topology from rendered images rather than from graph-native representations—was defined as part of that internship’s scope. Beyond the assigned scope, the problem area also aligned with the author’s existing interest in computer vision, which motivated a deeper engagement with the CNN-based approach and its failure modes than a purely assigned task would otherwise warrant. In practice, this meant that once the initial pipeline was working, additional time was spent specifically on the interpretability analysis in Section 8—probing *why* the model works rather than stopping at the headline accuracy numbers—since that question was of independent interest to the author beyond what the assigned deliverable strictly required.

3 Dataset Description

3.1 Synthetic Dataset Pool

The primary training, validation, and testing dataset consists of 2,500 synthetic graphs generated via NetworkX [1]. The dataset is stratified across 5 generator families crossed with 4 node-count tiers. Each cell in this generator $\times$ tier matrix contains exactly 125 graphs. The generator families and tiers were chosen deliberately rather than arbitrarily: each family instantiates a structurally distinct regime that a purely visual model might respond to very differently (uniformly random connectivity, scale-free hub formation, small-world clustering, strict tree hierarchies, and saturated density), while the four tiers ensure the model is exposed to a wide range of absolute node/edge counts rather than a narrow band where a crude heuristic could trivially succeed. Stratifying across both axes, rather than sampling freely, guarantees that no single tier or family dominates the training signal and that later breakdown analyses (Section 7) are comparing like-for-like sample sizes. The node range for each tier is defined as follows:

- **tiny:** 5–10 nodes
- **small:** 10–25 nodes
- **medium:** 25–50 nodes
- **large:** 50–100 nodes

The 5 generator families and their specific parameters from `config.yaml` are:

1. **Erdős–Rényi (erdos_renyi) [2]:** Random graphs with edge probability parameter $p \in \{0.05, 0.1, 0.2, 0.3\}$. Every edge is included independently at random, giving a structurally “featureless” baseline topology with no hubs and no clustering beyond what chance produces.

2. **Barabási–Albert** (`barabasi_albert`) [3]: Preferential attachment scale-free graphs with parameter $m \in \{1, 2, 3, 5\}$ representing the number of edges to attach from a new node to existing nodes. This family produces hub-and-spoke structures, which render visually as a small number of highly connected nodes surrounded by sparser satellites—a distinct visual signature from Erdős–Rényi graphs of the same size.
3. **Watts–Strogatz** (`watts_strogatz`) [4]: Small-world graphs with neighborhood size parameter $k \in \{4, 6\}$ and rewiring probability $p \in \{0.1, 0.3\}$. These graphs combine local clustering with a few long-range shortcuts, giving a ring-like visual layout punctuated by occasional cross-connections.
4. **Random Trees** (`random_tree`): Graph trees generated randomly using a deterministic spacing generator. Trees are the sparsest possible connected topology (exactly $n - 1$ edges), included specifically to test the model at the low-density extreme.
5. **Dense Erdős–Rényi variants** (`dense`): Denser random graphs with edge probability parameter $p \in \{0.6, 0.8, 0.95\}$. These sit at the opposite density extreme from the trees and, as later chapters show, turn out to be the *easiest* case for the CNN rather than the hardest, which is itself a notable finding rather than an expected one.

Each graph’s render and layout seed is derived deterministically from its `graph_id` via SHA-256, ensuring bit-for-bit reproducibility. The synthetic pool is split into train (70%, 1,750 graphs), validation (15%, 375 graphs), and test (15%, 375 graphs) splits, stratified by generator type, using seed 42.

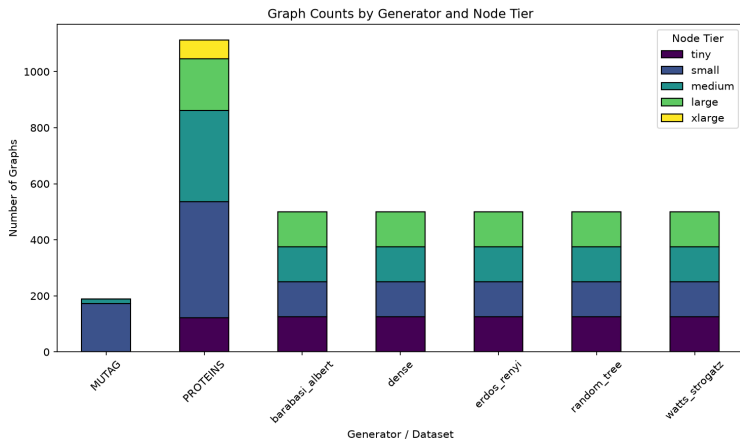


Figure 3.1. Distribution of graph counts across the synthetic size tiers.

3.2 Held-out Real-World Dataset

To test out-of-distribution generalization, we incorporate 1,301 real-world graphs loaded via PyTorch Geometric’s `TUDataset` [5]. These benchmarks were chosen because they are standard, widely-used graph-learning datasets from two different application domains (chemistry and biology), which lets us check whether anything the model learns from synthetic generator graphs transfers to structurally organic, human- or nature-produced topologies. The datasets are stripped of chemical/biological labels to expose only raw topology:

- **MUTAG (188 graphs) [6]:** A dataset of mutagenic aromatic and heteroaromatic nitro compounds. They are small and structurally homogeneous.
- **PROTEINS (1,113 graphs) [7]:** Contact maps representing secondary structure elements. They have a broad size distribution, spanning 4 to 620 nodes. Crucially, 67 of the PROTEINS graphs exceed the training ceiling of 100 nodes, serving as a severe size generalization test.

The held-out set was never seen during training or model selection, so any accuracy reported on it in Section 7 reflects genuine generalization rather than memorization of the training distribution.

3.3 Split and Crosstabulation Statistics

The exact per-split, per-tier, and per-generator counts pulled directly from the processed dataset CSVs are presented in Tables 3.1, 3.2, 3.3, and 3.4. These tables exist primarily as a stratification audit: because the splits are meant to be balanced across generator family and size tier, listing the raw counts lets a reader verify directly that no split is accidentally skewed toward one family or tier, which would otherwise silently bias the reported metrics. The corresponding structural distributions are visualized in Figure 3.2 and Figure 3.3.

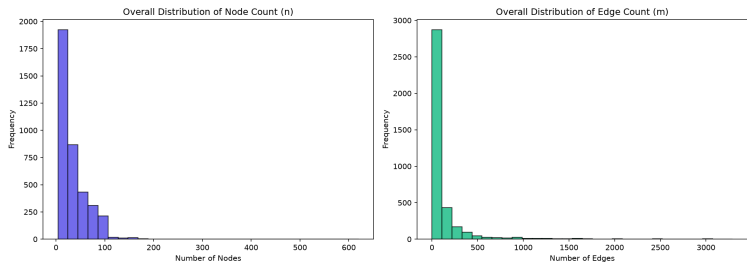


Figure 3.2. Overall node and edge count distributions across the dataset.

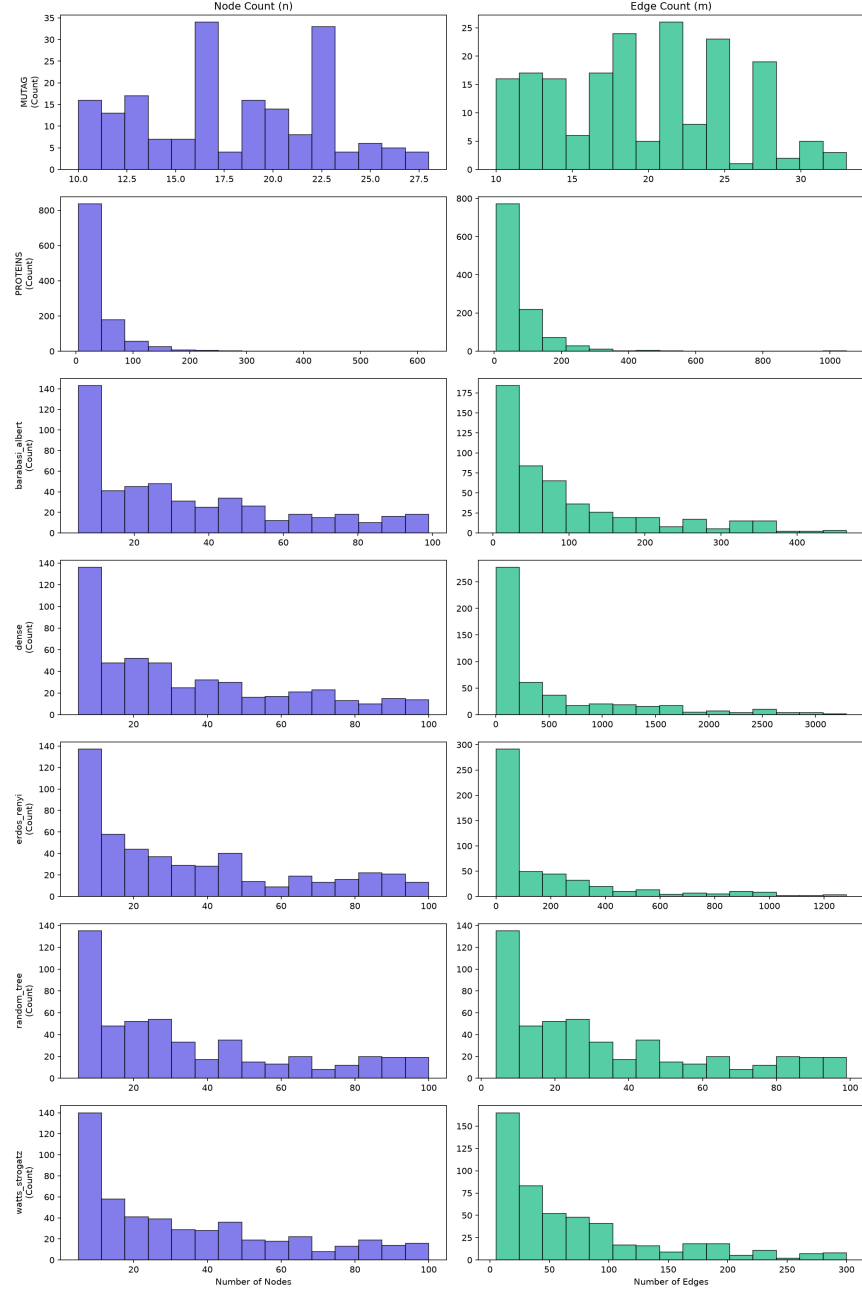


Figure 3.3. Node and edge distributions broken down by generator family.

Table 3.1. Train Split Sample Counts by Generator and Tier ($n = 1,750$)

Generator	Tiny (5–10)	Small (10–25)	Medium (25–50)	Large (50–100)	Total
barabasi_albert	89	92	92	77	350
dense	88	95	83	84	350
erdos_renyi	82	83	94	91	350
random_tree	92	86	89	83	350
watts_strogatz	91	89	88	82	350
Total	442	445	446	417	1,750

Table 3.2. Validation Split Sample Counts by Generator and Tier ($n = 375$)

Generator	Tiny (5–10)	Small (10–25)	Medium (25–50)	Large (50–100)	Total
barabasi_albert	22	13	16	24	75
dense	17	21	18	19	75
erdos_renyi	23	17	14	21	75
random_tree	15	22	18	20	75
watts_strogatz	20	14	21	20	75
Total	97	87	87	104	375

Table 3.3. Test Split Sample Counts by Generator and Tier ($n = 375$)

Generator	Tiny (5–10)	Small (10–25)	Medium (25–50)	Large (50–100)	Total
barabasi_albert	14	20	17	24	75
dense	20	9	24	22	75
erdos_renyi	20	25	17	13	75
random_tree	18	17	18	22	75
watts_strogatz	14	22	16	23	75
Total	86	93	92	104	375

Table 3.4. Held-out (Real-world) Split Sample Counts by Source and Tier ($n = 1,301$)

Source	Tiny (5–10)	Small (10–25)	Medium (25–50)	Large (50–100)	XLarge (> 100)	Total
MUTAG	0	173	15	0	0	188
PROTEINS	122	414	326	184	67	1,113
Total	122	587	341	184	67	1,301

4 Rendering Pipeline

Each graph is rendered to a 224×224 px PNG image at 100 DPI. This resolution was chosen as a standard, well-supported CNN input size that keeps training computationally tractable on the available hardware (Section 6) while still leaving enough pixel detail to distinguish individual nodes in the **large** tier. The layout is determined using a deterministic three-stage fallback chain:

`pygraphviz sfdp` $\longrightarrow$ `pydot sfdp` $\longrightarrow$ NetworkX spring layout

This chain exists for robustness rather than by design intent: **pygraphviz** is the preferred binding to Graphviz [8]’s force-directed **sfdp** layout engine, but it depends on a system-level Graphviz installation that is not guaranteed to be present on every machine the pipeline might run on (including, potentially, a grader’s or reviewer’s environment); **pydot** is a lighter-weight fallback that still calls out to the same underlying Graphviz binary, and NetworkX’s pure-Python spring layout is a last resort that requires no external dependency at all. In practice, all 3,801 rendered images (synthetic and real) successfully resolved through Graphviz **sfdp** (verified via the `layout_algorithm` field in `labels_processed.csv`), so the fallback tiers were never actually exercised—but their presence is what makes the rendering step reproducible across environments rather than silently failing. Figure 4.1 displays a representative sample render grid.

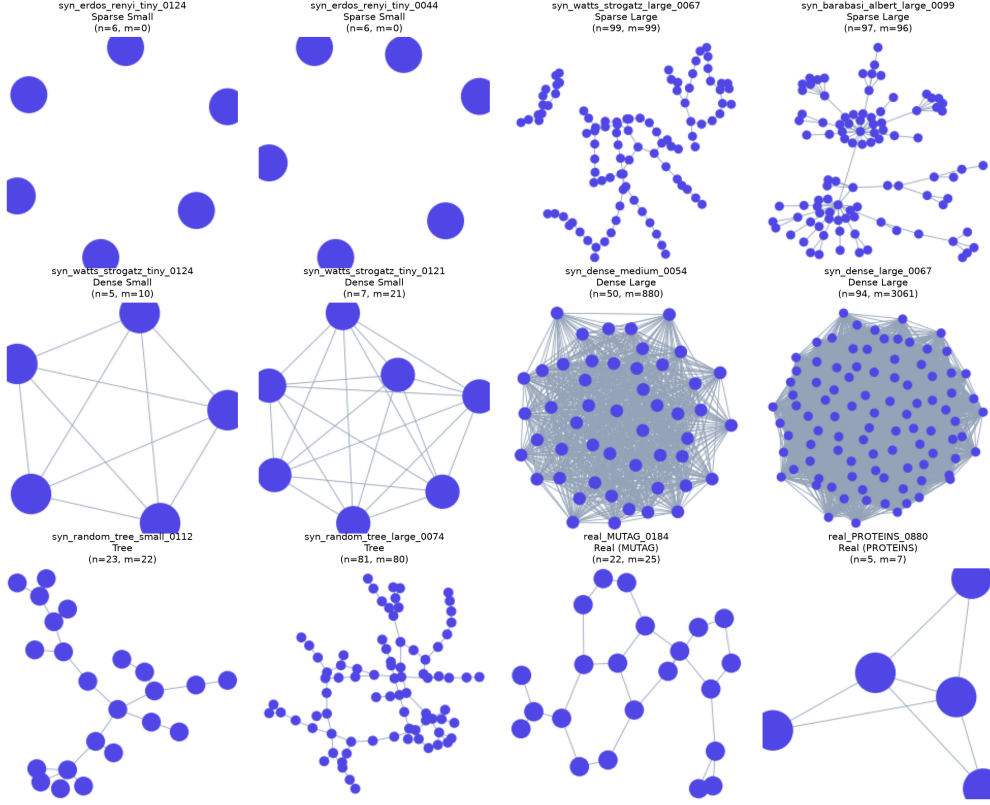


Figure 4.1. Sample rendered images from the dataset showcasing different topologies.

The visual style is defined by a solid white background, uniform node coloring (dark blue/purple), and fixed thin edges (width 0.6). Crucially, the node radius scales dynamically based on the vertex count n :

$$\text{radius} = \max\left(15, \frac{4000}{n}\right) \quad (4.1)$$

This formula exists to solve a real rendering problem: at fixed node size, a 100-node graph packed into a 224×224 image becomes a solid blob of overlapping circles with no visible internal structure, which would make the image useless to both the CNN and a human viewer. Shrinking nodes as n grows keeps the rendered graph visually legible across the full size range. However, this inverse size scaling introduces a significant confound: the model can potentially cheat by regressing the count directly from the physical size of the rendered nodes rather than counting individual nodes—since node radius and vertex count are, by construction, related by a fixed deterministic formula, a model does not need to understand topology at all to exploit it. This tension between legibility and confound-avoidance was recognized at design time and is analyzed empirically in Section 8, where it turns out to be a real and statistically significant effect rather than a hypothetical one.

5 Model Architecture

5.1 Custom CNN Regressor (CustomCNNRegressor)

The visual model is a custom deep convolutional neural network consisting of 4 convolutional blocks followed by a fully-connected regression head. The depth and channel width were chosen to match the scale of the problem: with only 2,500 synthetic training graphs, a very deep or very wide backbone (e.g. a full ResNet) would be prone to overfitting, while four blocks give enough receptive-field growth to see the whole 224×224 image by the final layer without requiring a dataset an order of magnitude larger to train reliably. Each convolutional block contains:

1. 3×3 Convolution (stride 1, padding 1)
2. 1D Batch Normalization
3. ReLU Activation
4. 2×2 Max-Pooling (stride 2)

The channel progression through the blocks is $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$, following the common convention of doubling channel width each time spatial resolution is halved, so that the total amount of information the network can represent stays roughly balanced across depth. Following the final block, the feature map is reduced using Global Adaptive Average Pooling to a size of $256 \times 1 \times 1$, rather than flattening and feeding a large feature vector into the regression head; average pooling was chosen specifically because vertex and edge counts should not depend on *where* in the image a node cluster happens to sit, and average pooling discards that positional information by construction while a flattened layer would not. The regression head contains:

$$\text{Linear}(256 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(128 \rightarrow 2)$$

producing both the vertex-count and edge-count predictions from a single shared trunk, with dropout included specifically to regularize this small-dataset regime.

5.2 GCN Baseline (GraphCountGCN)

The graph-native baseline consists of 3 Graph Convolutional Network [9] layers (GCNConv, implemented via PyTorch Geometric [10]) with progression: $1 \rightarrow 64 \rightarrow 64 \rightarrow 64$. Each layer is followed by Batch Normalization and a ReLU activation. The node embeddings are pooled globally using a mean-pooling layer (`global_mean_pool`) and fed through a fully-connected regression head identical in shape to the CNN's ($64 \rightarrow 128 \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(128 \rightarrow 2)$); keeping the two regression heads identical was a deliberate fairness choice, so that any performance gap between the two models reflects the encoder (convolution-on-pixels vs. message-passing-on-topology) rather than a difference in how predictions are ultimately produced. Crucially, the only input feature provided to each node is its normalized degree:

$$x_v = \frac{\deg(v)}{\max_{u \in V} \deg(u)}$$

This GCN represents a minimal topological learner with a highly restricted feature budget—no node identity, no positional encoding, no coordinates, nothing but a single scalar per node. This was a deliberate design choice to isolate the comparison to *topology alone* versus *topology as rendered*

geometry, rather than letting the GCN implicitly recover spatial information the CNN also has access to. It is also, as discussed in Section 9 and Section 10, the single biggest caveat on the headline CNN-vs-GCN comparison, since a GCN given richer features would likely close some of the gap.

5.3 Graph-Statistic Baseline

The heuristic baseline represents a non-learned control: a sanity floor that any trained model should be expected to beat, and a useful reference point for how much of the task is “solvable” by simple counting rules alone rather than requiring learning at all. For vertices, it returns the true count (oracle). For edges, it computes the estimate:

$$E_{\text{pred}} = \text{mean_density_per_generator} \times \frac{n(n-1)}{2}$$

where the per-generator mean densities are computed on the training set. Because this heuristic assumes every graph of a given generator type shares that generator’s average density, it should do reasonably well on graphs close to the population mean and poorly on outliers—a prediction borne out by the results in Section 7.

5.4 Log Target Transformation

Both continuous regression targets are transformed during training using:

$$y_{\text{train}} = \log(y_{\text{true}} + 1)$$

This transformation matters because the raw target distribution is extremely right-skewed: edge counts in the training data range from 5 up to 1,049, so a model trained directly on raw counts would be dominated by loss contributions from the handful of large, dense graphs, at the expense of learning to distinguish, say, a 6-node graph from an 8-node one. The log transform compresses that dynamic range so gradient magnitudes stay comparable across small and large graphs alike. At inference, predictions are back-transformed via $\exp(y_{\text{pred}}) - 1$, rounded to the nearest integer, and clamped at 0 (since a negative vertex or edge count is not meaningful).

6 Training Setup

Both models were trained using PyTorch [11] on a Google Colab environment with a T4 GPU, since the local development machine’s CPU-only setup made full training runs impractical within the project’s one-week timeline. The hyperparameters sourced from `config.yaml` are:

- **Optimizer:** Adam [12] (learning rate = 0.001)
- **Loss Function:** Mean Squared Error (MSE) on the log-transformed targets
- **Batch Size:** 32

- **Epochs:** 30
- **Reproducibility:** Random seeds fixed at 42 for split generation and initialization

These are standard, widely-used defaults for small-to-medium image and graph regression tasks rather than the result of an extensive hyperparameter search; given the project’s timeline, the priority was a stable, well-understood configuration over marginal tuning gains, and Adam with a 10^{-3} learning rate is a reasonable starting point for both a CNN and a GCN of this size. The best checkpoints were saved based on validation loss rather than the final epoch weights, which protects against reporting a model that has started to overfit the training set in its last few epochs. A retraining run revealed a GPU/cuDNN non-determinism effect, shifting test-split overall vertex MAE from 4.5 to 3.2 between otherwise identical runs; this is reported openly here (and revisited in the Limitations chapter) because it means the headline metrics in Section 7 should be read as one draw from a distribution of plausible outcomes for this configuration, not as an exactly reproducible constant.

7 Results

7.1 Baseline Comparison

The quantitative comparison results on the Test and Held-out splits are presented in Table 7.1. The CNN outperforms the GCN baseline on every single metric, achieving a $4\text{--}6\times$ lower error on the synthetic split and a $2\times$ lower error on the real held-out split. It also significantly outperforms the density heuristic. In practical terms, a vertex MAE of 3.20 on the synthetic test split means the CNN’s typical prediction is within three nodes of the true count across graphs ranging from 5 to 100 nodes—a small error relative to the scale of the task, though as later chapters show, that error grows substantially on the largest and most out-of-distribution graphs rather than staying uniform. The GCN’s much larger errors are consistent with the deliberately minimal, degree-only feature set described in Section 5: with no richer structural signal to draw on, it has comparatively little to work with beyond local connectivity patterns.

7.2 Detailed Breakdown Reports

Detailed performance breakdowns by generator type (test split), source dataset (held-out split), and density bucket (held-out split) are shown in Tables 7.2, 7.3, and 7.4. These breakdowns exist to check whether the aggregate numbers in Table 7.1 are representative or are instead being carried by one easy subgroup while others perform far worse. Predictions on the test split are visually compared in Figure 7.1a and Figure 7.1b, and predictions on the held-out split are analyzed in Figure 7.2a and Figure 7.2b.

Table 7.1. Baseline Comparison Metrics on Test and Held-out Splits

Split	Target	Metric	CNN	GCN	Graph-statistic
Test (Synthetic)	Vertices	MAE	3.20	19.05	0.00*
	Vertices	RMSE	6.08	26.61	0.00*
	Edges	MAE	25.39	148.98	120.53
	Edges	RMSE	81.75	391.67	233.51
Held-out (Real)	Vertices	MAE	10.62	21.15	0.00*
	Vertices	RMSE	31.30	44.09	0.00*
	Edges	MAE	28.03	56.84	363.65
	Edges	RMSE	67.72	92.82	2155.29

* The graph-statistic baseline’s 0.0 vertex MAE is structurally trivial (it returns the true node count directly). Only its edge heuristic is a genuine prediction.

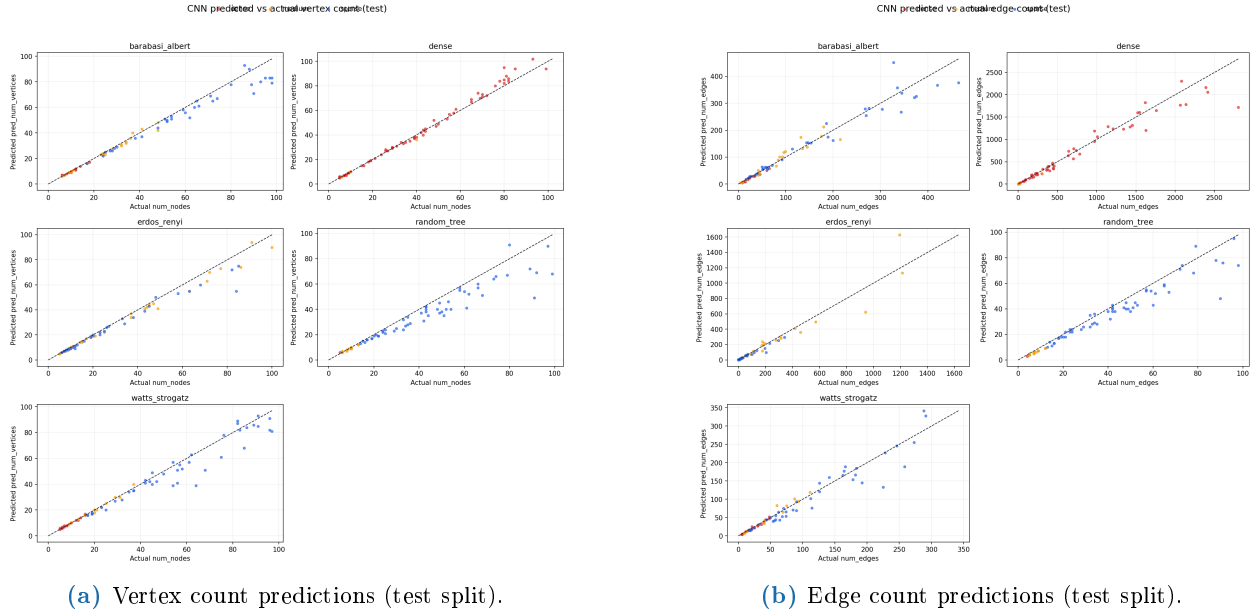
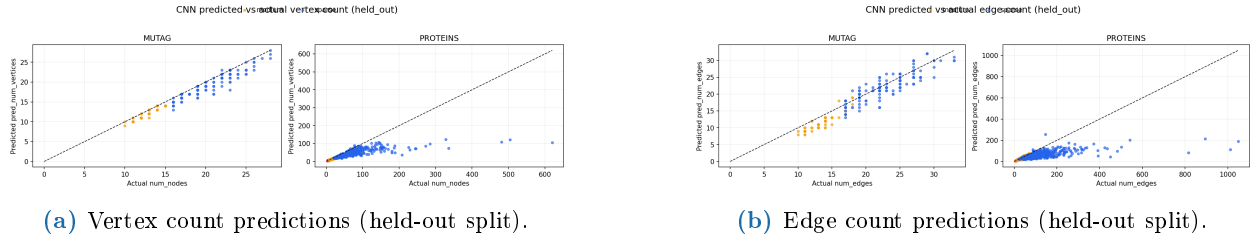
**Figure 7.1.** Scatter plots of predictions vs. ground truth on synthetic test data.**Figure 7.2.** Scatter plots of predictions vs. ground truth on real held-out data.

Table 7.2. Detailed MAE by Generator Type on the Test Split

Generator	CNN Vertices	GCN Vertices	CNN Edges	GCN Edges	Graph-stat Edges
barabasi_albert	2.83	18.68	13.80	81.53	184.23
dense	1.75	21.65	75.63	497.71	76.67
erdos_renyi	2.47	15.49	22.48	96.92	51.83
random_tree	5.69	17.71	4.77	17.60	87.81
watts_strogatz	3.27	21.71	10.27	51.15	202.09

Table 7.3. Detailed MAE by Dataset on the Held-out Split

Dataset	CNN Vertices	GCN Vertices	CNN Edges	GCN Edges	Graph-stat Edges
MUTAG	1.07	5.45	1.85	17.03	24.60
PROTEINS	12.23	23.80	32.45	63.57	420.93

Table 7.4. Detailed MAE by Density Bucket on the Held-out Split

Density Bucket	CNN Vertices	GCN Vertices	CNN Edges	GCN Edges	Graph-stat Edges
dense	0.43	13.94	1.13	72.10	6.51
medium	2.03	7.57	5.91	44.96	12.63
sparse	18.75	32.13	49.11	61.90	683.54

Two patterns stand out across these breakdowns and are followed up in detail in Section 8: first, the CNN’s vertex error is lowest specifically on **dense** graphs and highest on **sparse** ones (Table 7.4)—the opposite of what one might naively expect if clutter were the main difficulty, and a pattern that turns out to be connected to the node-size rendering convention rather than to genuine ease of counting. Second, the graph-statistic heuristic’s edge error explodes specifically on **sparse** graphs (683.54), because a single fixed per-generator density estimate is a poor fit for the tail of unusually sparse instances within a family—exactly the failure mode a non-learned heuristic baseline is expected to exhibit.

8 Model Interpretation & Failure Analysis

Section 7 answered the Competitiveness question from the Introduction: the CNN beats the GCN baseline on every metric. This chapter exists to answer the harder Mechanism question—*why*—since a strong accuracy number alone cannot distinguish a model that has learned genuine structural reasoning from one that has found an easier shortcut that merely correlates with the correct answer on this particular rendering pipeline. Each section below is a separate, targeted probe designed to rule in or rule out a specific candidate explanation, moving from a qualitative look at what the network attends to (Grad-CAM), to a direct causal intervention on the strongest suspected confound (the node-size probe), to statistical significance testing of that intervention, to an alternative low-level explanation (pixel counting), to a systematic characterization of where errors concentrate, and finally to an end-to-end validation against the deployed application itself.

8.1 Grad-CAM Spatial Attributions

Grad-CAM [13] heatmaps are computed at the final convolutional layer (`features[3]`, resolution 14×14) as a first, qualitative pass at the Mechanism question: before running any targeted intervention, it is worth simply visualizing where in the image the network’s gradients concentrate, since a model attending to background canvas or to arbitrary corners would already be a red flag before any quantitative probe is needed.

- **Attribution Focus:** Activation is concentrated on the spatial locations of node clusters rather than the background canvas, as shown in the generator grid (Figure 8.1). This is a reassuring baseline sanity check—the model is at least looking at the graph rather than at rendering artifacts elsewhere in the frame.
- **Edge vs. Vertex Attributions:** The vertex-target maps are tightly localized around individual node boundaries, whereas the edge-target maps are diffuse, covering the structural gaps between nodes. This distinction is intuitive: counting vertices is a localized, per-blob task, while counting edges requires attending to the connective tissue between blobs, so the two output heads would be expected to draw on different spatial evidence even though they share the same convolutional trunk.
- **Failure Activation:** Worst-error cases exhibit activation concentrated on a single cluster rather than spreading across the entire graph boundary, revealing a spatial attention bottleneck (visualized in Figure 8.2)—consistent with the size-generalization failures quantified later in Section 8.6, where large graphs that do not fit the network’s learned notion of “how big a graph looks” produce disproportionately large errors.

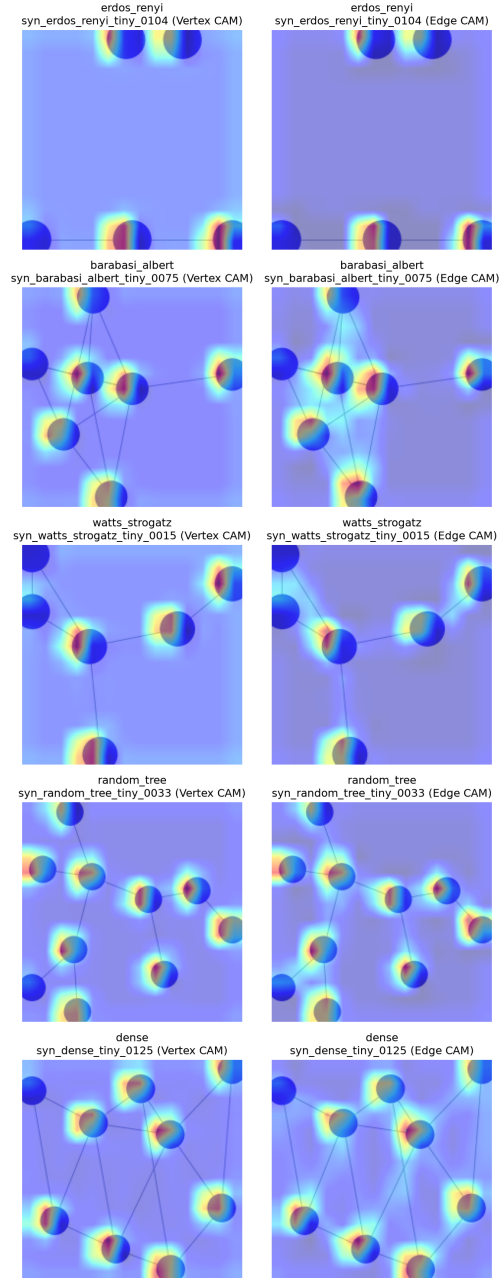


Figure 8.1. Grad-CAM activations mapped across representatives of each generator family.

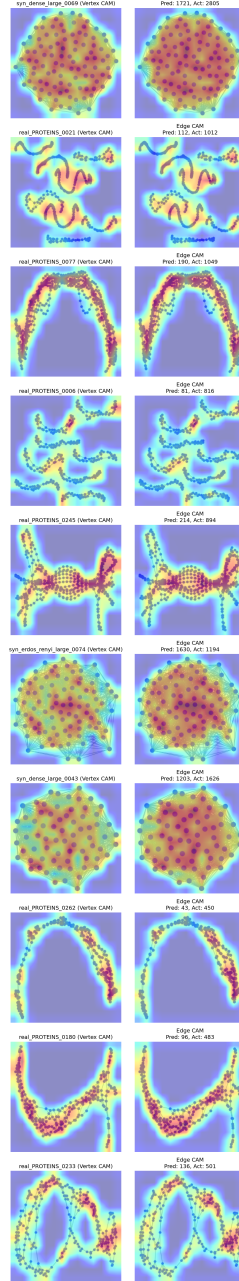


Figure 8.2. Grad-CAM activations on high-error failure cases.

8.2 Shortcut-Learning Probe (Node-Size)

The rendering formula in Section 4 (Equation 4.1) was flagged at design time as a potential shortcut, since node radius is a deterministic function of vertex count and therefore a trivially exploitable proxy. This section tests that hypothesis directly rather than leaving it as a hypothesis. To test the node-size shortcut directly, we re-rendered a probe set of 40 graphs (10 per tier) with a constant node size, holding layout positions and edge widths fixed—if the model’s accuracy depends on the true visual size cue rather than on genuine topological reasoning, removing it while holding everything

else constant should degrade accuracy measurably. Examples of these probe variants are visualized in Figure 8.3, and the tier-wise MAE comparisons are plotted in Figure 8.4. Table 8.1 displays the numeric results. The overall vertex MAE triples from 3.425 to 10.475 when the size cue is removed, confirming that the model heavily exploits node-size inverse scaling.

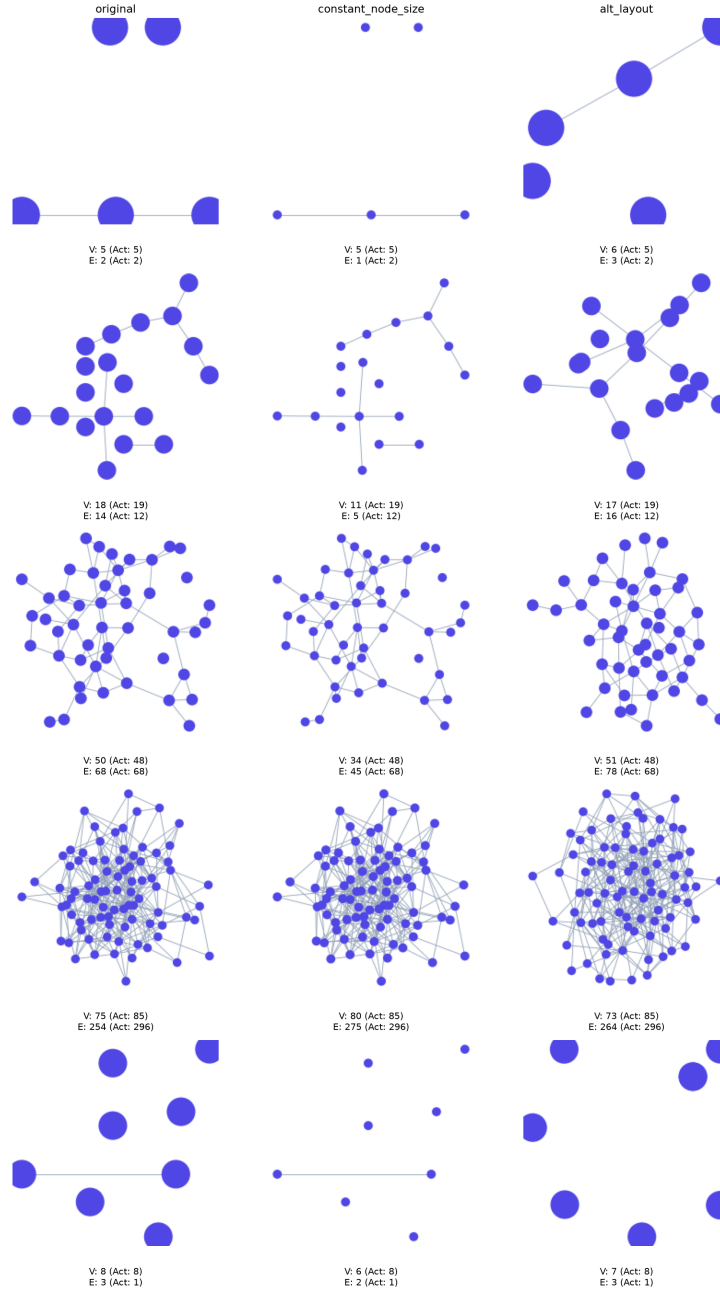


Figure 8.3. Side-by-side comparison of original, constant node-size, and alternative layouts.

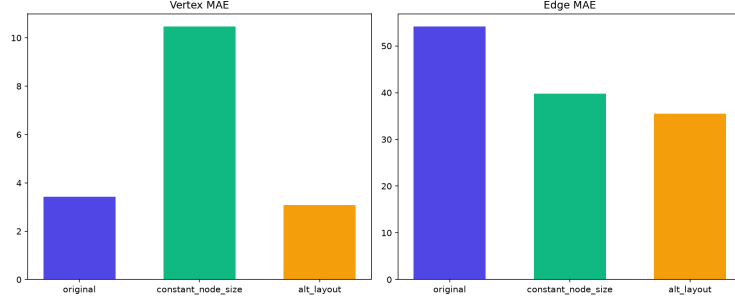


Figure 8.4. Error comparison (MAE) across the different probe variants.

Table 8.1. Probe Summary MAE by Variant and Size Tier

Tier	Orig. Vert.	Fixed Vert.	Alt Vert.	Orig. Edges	Fixed Edges	Alt Edges
tiny	0.3	1.4	0.4	1.6	3.5	1.6
small	1.0	8.5	1.0	3.3	21.2	4.9
medium	3.5	15.3	3.8	16.3	51.7	13.2
large	8.9	16.7	7.1	195.5	82.8	122.4
Overall	3.425	10.475	3.075	54.175	39.8	35.525

8.3 Layout-Sensitivity Probe

The node-size probe isolates one candidate shortcut, but layout geometry itself—the specific (x, y) coordinates `sfdp` happens to produce—is a second plausible confound, since the CNN could in principle be keying off Graphviz-specific spatial patterns rather than the topology those patterns represent. We re-rendered the 40 probe graphs using a Kamada-Kawai layout instead of Graphviz `sfdp`. The results (Table 8.1) show that the model’s accuracy remains stable, indicating that the CNN is layout-insensitive within this family and did not overfit to Graphviz-specific coordinates.

8.4 Wilcoxon Signed-Rank Tests

The mean-MAE comparisons above are suggestive but vulnerable to distortion by a handful of outlier graphs, as the edge-count results in this very section demonstrate. A paired Wilcoxon signed-rank test [14] is used instead because it is a non-parametric test on the direction and rank of per-graph differences rather than their raw magnitude, making it far more robust to exactly this kind of outlier distortion, and because the probe design (each of the 40 graphs rendered under two conditions) naturally produces paired observations. The test ($n = 40$) confirms that removing the node-size cue significantly degrades vertex prediction ($p = 3.5799 \times 10^{-6}$, 33/40 graphs worse) and edge prediction ($p = 2.9778 \times 10^{-4}$, 35/40 graphs worse). In contrast, the layout intervention shows a clean null result ($p = 0.4991$ for vertices, $p = 0.8978$ for edges), validating that layout change does not affect accuracy significantly (see Table 8.2). Together, these two tests separate the two candidate confounds cleanly: node size is a real, statistically significant driver of accuracy, while layout geometry is not.

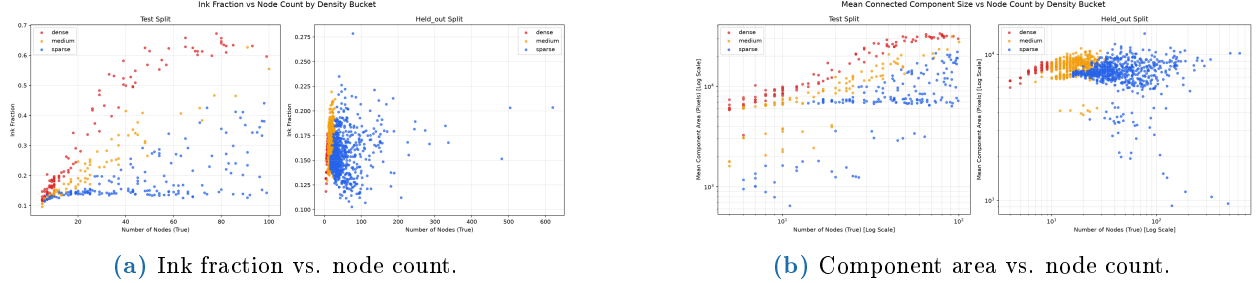
Table 8.2. Wilcoxon Signed-Rank Test Results for Node-Size and Layout Probes ($n = 40$)

Probe Intervention	Target	Original Mean (Median)	Intervention Mean (Median)	Worse/Better/Tie	Wilcoxon Stat	p -value	Signif. (0.05)
Fixed Node Size	Vertices	3.425 (1.0)	10.475 (11.0)	33 / 4 / 3	44.5	3.58×10^{-6}	Yes
	Edges	54.175 (3.5)	39.8 (20.0)	35 / 5 / 0	141.0	2.98×10^{-4}	Yes*
Alt. Layout (KK)	Vertices	3.425 (1.0)	3.075 (1.0)	17 / 16 / 7	243.5	0.4991	No
	Edges	54.175 (3.5)	35.525 (4.5)	18 / 16 / 6	290.0	0.8978	No

* Although the mean error decreases due to outliers, the Wilcoxon test ranks confirm a significant worsening pattern for the 35/40 majority.

8.5 Ink-Coverage Correlational Analysis

The node-size probe rules in one confound, but a simpler explanation is still worth ruling out before crediting the model with any structural understanding at all: perhaps the CNN is not reasoning about discrete nodes and edges in any meaningful sense, but is simply performing regression against the total amount of ink on the page. We correlated predicted/true targets against low-level pixel statistics: `ink_fraction` (non-background pixels) and `mean_component_area`. Visual scatter diagrams of these relationships are provided in Figure 8.5a and Figure 8.5b. The resulting correlation statistics are shown in Table 8.3.

**Figure 8.5.** Correlations between visual pixel properties and actual node counts.**Table 8.3.** Pearson Correlations (r) for Low-Level Pixel Statistics vs. True and Predicted Graph Properties

Metric X	Metric Y	Test Split r	Test Split p	Held-out Split r	Held-out Split p
ink_fraction	num_nodes	0.5378	1.72×10^{-29}	0.0700	0.0115
ink_fraction	num_edges	0.8235	7.48×10^{-94}	0.0930	7.84×10^{-4}
ink_fraction	pred_num_vertices	0.6182	6.59×10^{-41}	0.1722	4.04×10^{-10}
ink_fraction	pred_num_edges	0.8453	1.49×10^{-103}	0.4654	6.58×10^{-71}
mean_component_area	num_nodes	0.5521	2.66×10^{-31}	-0.0830	0.0027
mean_component_area	pred_num_vertices	0.6272	2.10×10^{-42}	0.0010	0.9705
num_components	num_nodes	-0.2403	2.52×10^{-6}	0.3081	5.29×10^{-30}

On the test split, ink fraction correlates strongly with predicted edge count ($r = 0.8453$). A linear baseline fit of `ink_fraction` to `num_edges` explains 82% of the variance ($R^2 = 0.8192$, MAE=83.73). On the held-out split, however, this correlation collapses completely ($r = 0.0930$, $R^2 = 0.0035$, MAE=47.25), proving that the CNN is not merely counting pixels to generalize to real graphs. An ink-fraction measurement inconsistency exists between the two analytical scripts: one uses `white_threshold=250` ($r = 0.8235$) and the other uses `bg_threshold=200` ($r = 0.9051$).

8.6 Failure Taxonomy Analysis

The probes above establish *that* shortcuts exist; this section asks *where* the model actually fails in practice, which matters for anyone deciding whether to trust a given prediction. Graphs are categorized into three bins: `out_of_distribution_size` ($N > 100$), `high_density_clutter` (density ≥ 0.40), and `in_distribution_normal` (density < 0.40). The errors are plotted as functions of size and density in Figure 8.6a and Figure 8.6b. The categorization summary is given in Table 8.4. Size extrapolation is the dominant failure mode, producing catastrophic error rates (mean vertex MAE of 96.64). Conversely, density-driven visual clutter is not a failure mode; `high_density_clutter` graphs produce the lowest errors (mean vertex MAE of 0.43)—this is the same pattern already visible in the density-bucket breakdown in Section 7, and is consistent with the node-size shortcut identified above: dense graphs at a given node count render with small, sharply-scaled nodes (via Equation 4.1), giving the model a strong, unambiguous size cue to exploit, whereas sparse and especially oversized graphs fall outside the range of node sizes seen during training. Images that produced the highest overall prediction errors are displayed in Figure 8.7.

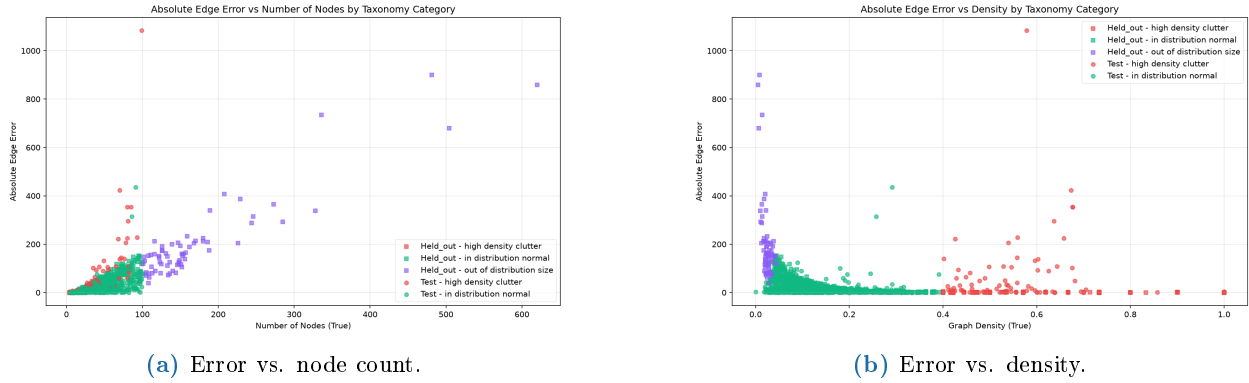


Figure 8.6. Plotting prediction error against physical graph size and structural density.

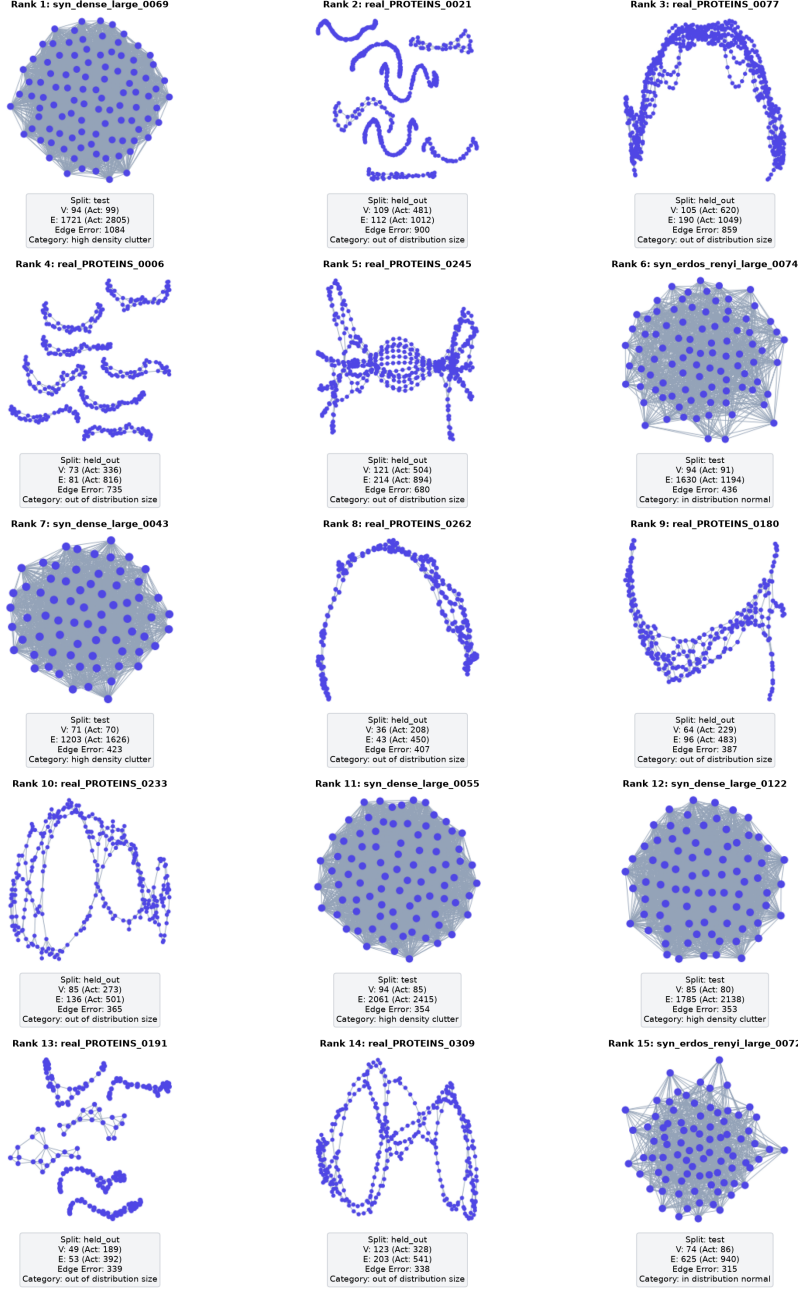


Figure 8.7. Grid of worst-performing outlier images yielding highest regression errors.

Table 8.4. Failure Taxonomy Error Analysis (CNN Regressor)

Split	Category	Mean V. MAE	Med. V. MAE	Mean E. MAE	Med. E. MAE	n
Held-out (Real)	High-density clutter	0.43	0.0	1.13	1.0	144
	In-distribution (normal)	6.67	3.0	20.79	10.0	1090
	Out-of-distribution (size)	96.64	71.0	203.60	155.0	67
Test (Synthetic)	High-density clutter	1.27	1.0	51.82	4.0	110
	In-distribution (normal)	4.00	2.0	14.42	3.0	265

8.7 Structural Attribute Correlations

Beyond size and density, it is worth checking whether other graph-theoretic properties are themselves confounded with the error patterns above, since an unexamined correlate could offer an alternative (or complementary) explanation for the failure taxonomy. Evaluating other graph features yields additional insights:

- **Average Clustering Coefficient:** Highly collinear with graph density ($r = 0.8362$ test, $r = 0.5963$ held-out), thus providing redundant signal—it does not add new explanatory power beyond what density already captures.
- **Graph Diameter vs. Error:** Independent of size ($r = 0.3364$ test). The partial correlation between diameter and vertex error (controlling for node count) flips sign: positive in synthetic ($r = +0.4777$) and negative in real ($r = -0.2867$). This sign flip is a caution against over-interpreting diameter as a stable predictor of error; it behaves differently between the synthetic and real distributions and should not be treated as a reliable signal in either direction.

8.8 Warning Banner Validation

The deployed application surfaces a risk warning to users based on graph density, and this section exists to audit whether that user-facing claim is actually supported by the data rather than simply assumed. A size-binned validation of the single flat dense-bucket warning (MAE citation 0.43/1.12) is shown in Table 8.5. For vertices, dense graphs are safer than sparse graphs at all sizes (median error 5.0 vs 21.0 in the 75–100 node bin). For edges, density and size interact: dense graphs are tied at small sizes but degrade sharply, showing 3.3 $\times$ higher median error by 75–100 nodes (224.0 vs 67.5). The flat warning banner is therefore miscalibrated and should be split into a size-aware warning.

8.9 Live-App Validation Panel

Every probe so far has run against curated offline splits. This final validation panel exists to check that the same conclusions hold when predictions are generated through the actual deployed inference path—the Predict page a real user would interact with—rather than only in a controlled evaluation script, since a discrepancy between the two would undermine the practical relevance of everything above it.

Table 8.5. Warning Banner Size-Binned Error Validation

Size Bin	Target	Sparse Mean (Med.)	Dense Mean (Med.)	Mean Delta	Group Size (D/S)
0–15 nodes	Vertices	0.53 (0.0)	0.39 (0.0)	-0.14	199 / 17
	Edges	1.47 (1.0)	1.47 (1.0)	0.00	
15–30 nodes	Vertices	2.63 (2.0)	0.56 (0.5)	-2.07	16 / 251
	Edges	6.80 (3.0)	13.00 (8.0)	6.20	
30–50 nodes	Vertices	8.27 (8.0)	1.41 (1.0)	-6.86	17 / 265
	Edges	26.82 (27.0)	51.47 (48.0)	24.66	
50–75 nodes	Vertices	14.37 (13.0)	2.18 (2.0)	-12.19	11 / 159
	Edges	43.45 (42.0)	122.82 (105.0)	79.37	
75–100 nodes	Vertices	23.33 (21.0)	6.18 (5.0)	-17.15	11 / 90
	Edges	68.59 (67.5)	285.00 (224.0)	216.41	

8.9.1 Part 1: Deployed App Spot Check

Table 8.6 documents a 23-graph spot check on the Predict page, covering a mix of synthetic generator types and real MUTAG/PROTEINS graphs so that both in-distribution and out-of-distribution behavior are represented in the same table. A critical observation is the extreme edge overestimation under combined dense and large conditions (such as `syn_dense_large_0069`, where true edge count is 2,805 but GCN predicts 93 and CNN predicts 1,721). Furthermore, GCN size generalization is worse than the CNN’s on the OOD-size cases (mean GCN vertex error 237 vs. CNN 179, $n = 6$).

Table 8.6. Predict Deployed App Spot Check Results ($n = 23$)

Index	File Name	True V	True E	CNN V	CNN E	GCN V	GCN E	Baseline V	Baseline E
1	syn_erdos_renyi_large_0088	63	309	55	303	23	54	63	536
2	syn_barabasi_albert_large_0044	54	200	51	162	35	60	54	393
3	syn_watts_strogatz_small_0031	13	39	12	37	17	73	13	21
4	syn_random_tree_tiny_0112	8	7	8	6	23	32	8	8
5	syn_dense_large_0069	99	2805	94	1721	20	93	99	1332
6	syn_erdos_renyi_small_0113	15	16	14	16	19	16	15	29
7	syn_watts_strogatz_large_0057	56	112	51	102	28	81	56	423
8	syn_dense_medium_0067	44	436	45	468	27	152	44	260
9	syn_dense_large_0016	58	973	61	956	27	164	58	454
10	real_MUTAG_0026	13	13	13	10	12	17	13	21
11	real_MUTAG_0004	19	22	18	21	13	35	19	47
12	real_PROTEINS_0964	26	47	20	29	19	48	26	89
13	real_PROTEINS_0820	22	44	20	30	28	51	22	63
14	real_PROTEINS_0484	38	79	25	31	29	59	38	193
15	real_PROTEINS_0274	105	178	78	107	32	41	105	1499
16	real_PROTEINS_0586	134	200	91	128	27	34	134	2446
17	real_PROTEINS_0236	116	255	40	43	20	42	116	1831
18	real_PROTEINS_1001	5	9	6	7	11	32	5	3
19	real_PROTEINS_0706	7	14	7	15	31	115	7	6
20	real_PROTEINS_0143	99	156	76	100	21	26	99	1332
21	real_PROTEINS_0043	101	220	71	101	23	52	101	1386
22	real_PROTEINS_0245	504	894	121	214	18	14	504	34792
23	real_PROTEINS_0077	620	1049	105	190	36	44	620	52670

8.9.2 Part 2: Novel-Image Sanity Check

The spot check above still draws from images generated by the project’s own rendering pipeline. This second check pushes further out-of-distribution by feeding the deployed app images it has never been trained or evaluated on in any form—scans, hand-drawings, and stylistically different diagrams—to see how gracefully (or not) the pipeline degrades outside its native rendering style. We validated the app’s pipeline using 7 custom images not containing ground-truth topology (Table 8.7). The errors scale based on visual discrepancy relative to the Graphviz training style.

Table 8.7. Predict App Novel-Image Sanity Check Results

Index	Image Style Details	Pred V	Pred E	Max Possible Edges	Plausible?
1	Uncropped notebook page, graphs + prose	5	19	10	No (Violation)
2	Scanned academic paper figure	93	746	4278	Yes
3	Clean vector lattice diagram	16	63	120	Yes
4	Hand-drawn network (rectangle nodes)	13	62	78	Yes
5	Cartoon-icon diagram	20	104	190	Yes
6	Weighted directed graph (source/sink)	474	15912	112101	Yes
7	Vector graph (numbered circular nodes)	32	74	496	Yes

Note: “Plausible?” indicates whether the predicted edge count is combinatorially plausible for the predicted vertex count, i.e. $E_{\text{pred}} \leq V_{\text{pred}}(V_{\text{pred}} - 1)/2$.

8.9.3 Part 3: Combinatorial Plausibility Violations

Because the CNN predicts vertex and edge counts as two independent regression outputs from a shared trunk, nothing in the architecture explicitly enforces that the two predictions remain mutually consistent; this final check is a cheap, purely arithmetic sanity test of whether the two output heads still respect basic graph theory in practice, without requiring any additional ground truth. A combinatorial check verifies whether predicted edge counts satisfy the theoretical limit:

$$E_{\text{pred}} \leq \frac{V_{\text{pred}}(V_{\text{pred}} - 1)}{2} \quad (8.1)$$

This condition holds across all 1,676 offline test/held-out predictions. However, during the novel image check, the uncropped notebook photo containing two diagrams and handwritten prose violated this bound ($V_{\text{pred}} = 5$, $E_{\text{pred}} = 19$, max possible = 10). This indicates that non-graph text artifacts break the visual assumptions of the regression layer.

9 Discussion

This chapter steps back from the individual results in Sections 7 and 8 to ask what they mean collectively, revisiting the two research questions posed in the Introduction: Competitiveness (can a

CNN match or beat a graph-native GCN on this task?) and Mechanism (if so, is it doing something resembling structural reasoning, or exploiting rendering artifacts?).

9.1 On Competitiveness

The primary research question is answered affirmatively, but only within the specific setup tested here: under a controlled visual rendering pipeline, an ordinary image-based CNN achieves lower error rates than a graph-native GCN baseline on both the synthetic and real-world splits. The CNN outperformed the GCN by 4–6 $\times$ on synthetic splits and by 2 $\times$ on held-out real splits. This is a genuinely useful result for the practical scenario motivating this project—recovering structure from an image when no graph-native representation is available—since it demonstrates that a purely visual pipeline is not merely a fallback of last resort but can be competitive with, or better than, a straightforward graph-native learner. However, the comparison requires two qualifications substantial enough that they change how the headline number should be read, not just minor caveats attached after the fact.

9.2 On Mechanism

The interpretability work in Section 8 was designed specifically to answer whether the CNN’s advantage reflects real topological reasoning, and the evidence is mixed rather than clean in either direction. On one hand, a substantial portion of the CNN’s performance is driven by shortcut learning. The Wilcoxon signed-rank test confirmed that when the inverse node-size scaling confound is removed, vertex prediction error increases threefold (from a median of 1.0 to 11.0). The CNN is, at least in part, reading topology off of visual rendering conventions (node radius) rather than performing abstract counting from first principles. This is corroborated by the failure taxonomy: the CNN’s best-performing category is `high_density_clutter`—graphs that render with small, sharply size-scaled nodes—while its worst category by a wide margin is `out_of_distribution_size`, where the node-size cue has never been observed at that scale during training. If the model were purely counting visual blobs independent of the renderer’s size convention, this particular pattern (dense graphs easiest, large graphs catastrophically hard) would not be the expected shape of the error distribution.

On the other hand, two pieces of evidence argue against dismissing the CNN’s accuracy as pure pixel-counting. First, the layout-sensitivity probe found a clean null result: switching from Graphviz `sfdp` to a Kamada-Kawai layout did not significantly change accuracy, indicating the model has not simply memorized `sfdp`-specific coordinate patterns. Second, and more tellingly, the pixel-only baseline regression shows that the ink-fraction-to-edge-count correlation that explains 82% of variance on synthetic data collapses almost entirely on real held-out data ($r = 0.8453 \rightarrow r = 0.0930$), yet the CNN’s own accuracy on real data remains reasonably good by comparison to both baselines. If the network were relying purely on total ink coverage as a proxy for edge count, its real-world accuracy should have collapsed in step with that correlation—it did not, which suggests the CNN has learned something more structured than a single low-level pixel statistic, even if that “something” still leans heavily on the node-size convention identified above. Grad-CAM’s spatial attributions are consistent with this middle-ground picture: attention concentrates on node clusters (not background) and separates sensibly between vertex-relevant and edge-relevant regions, but on the worst failure cases collapses onto a single cluster rather than spanning the whole graph—a bottleneck that lines up with exactly the size-generalization failures the taxonomy identifies quantitatively.

Taken together, the most defensible reading is that the CNN has learned a hybrid strategy: it exploits the node-size rendering shortcut where available (which is most of the in-distribution data), but retains enough sensitivity to actual visual structure that its accuracy does not fully collapse when the simplest possible explanation (ink coverage) is ruled out. This is a more nuanced conclusion than either “the CNN learned real topology” or “the CNN is cheating,” and it is the reason this project treats the interpretability chapter as a co-equal deliverable alongside the headline accuracy numbers rather than as a follow-up appendix.

9.3 On the Fairness of the Comparison

Second, the GCN baseline was highly constrained. Funneling a graph structure through a layout generator like Graphviz implicitly solves a portion of the topological complexity for free (representing centrality, density, and local components as spatial clustering). The CNN receives this layout coordinate mapping implicitly. A GNN equipped with positional encodings, spectral features, or richer node attributes would likely bridge the performance gap. This means the Competitiveness result in Section 9.1 should be read narrowly—as evidence that a CNN can beat *this particular*, deliberately minimal GCN—rather than as a general claim that visual inference outperforms graph-native inference as a category. The Baseline-comparison-fairness item in the Limitations chapter expands on why this distinction matters for anyone tempted to over-generalize the headline result.

10 Limitations

The Discussion chapter argued for a nuanced, hybrid reading of the CNN’s success. This chapter enumerates the specific, concrete boundaries of that success in detail, so that the headline numbers in Section 7 are never read in isolation from the conditions under which they were obtained. Each item below was either identified as a risk at design time and subsequently confirmed or refuted empirically, or surfaced unexpectedly during the interpretability analysis in Section 8; none are purely hypothetical concerns.

- **Rendering and layout sensitivity:** All reported results are specific to Graphviz’s `sfdp` layout algorithm, which produced 100% of the training, test, and held-out images. The layout-sensitivity probe found no significant effect from switching to Kamada-Kawai on the small probed sample ($p = 0.499$ vertex, $p = 0.898$ edge), which is a reassuring but narrow result—it does not establish robustness to layout algorithms with substantially different visual conventions (e.g. hierarchical/Sugiyama-style layouts, circular layouts, or force-directed layouts with different edge bundling), none of which were tested.
- **Dataset scale and synthetic-to-real distribution gap:** The synthetic training set is 2,500 graphs capped at 100 nodes; no synthetic graphs above this ceiling were generated or evaluated, so the model’s behavior above 100 nodes is characterized only through the real-world PROTEINS subset (67 graphs, up to 620 nodes) and the 7-image live novel-image check, both of which show substantial and increasing overcounting well beyond the ceiling (up to $\sim 4.7\times$ on the most unfamiliar test image). This is a real generalization gap, not merely an untested edge case.

- **Image resolution, node-size tradeoff, and the confirmed shortcut:** Images are rendered at a fixed 224×224 px with node radius scaling as $\max(15, 4000/n)$. This choice was flagged as a potential shortcut at design time and is now empirically confirmed as a real, statistically significant contributor to the CNN’s vertex-count accuracy (formalized via Wilcoxon signed-rank test: median vertex error increase $1.0 \rightarrow 11.0$ under constant node size, $p = 3.5799 \times 10^{-6}$, 33/40 graphs worse). Reported accuracy figures should be read as partly attributable to this rendering convention and would likely be lower under a fixed-node-size rendering scheme. The edge-count comparison shows the same direction of effect via median and Wilcoxon test ($p = 2.98 \times 10^{-4}$) despite a misleading mean-based signal driven by two outlier graphs—report medians and win/loss counts for this specific comparison, not means.
- **Out-of-distribution size ceiling:** The 100-node training ceiling is the single dominant failure mode identified in this project, outweighing density or visual clutter, both in the offline failure taxonomy and in the live novel-image check. Practically, this means the current model should not be trusted on graphs suspected to exceed roughly 100 nodes without an explicit warning to the user, regardless of how clean or well-rendered the image otherwise looks—the failure here is a property of the training distribution’s size range, not of image quality.
- **Dense-bucket warning miscalibration:** The deployed app’s current warning banner treats “dense” as a single flat-risk bucket (MAE citation 0.43/1.12). Size-binned validation shows this is safe—in fact the safest bucket—for vertex prediction at every size tested; safe for edge prediction only at small sizes; and high-risk for edge prediction at large sizes ($3.3\times$ worse than sparse by 75–100 nodes). A size-aware warning split is recommended over the current flat citation.
- **Baseline-comparison fairness:** The GCN baseline’s only node feature is normalized degree—no richer structural encodings were provided. The headline CNN-vs-GCN result should be read as specific to this minimal-feature GCN, not as a general claim that image-based inference outperforms graph-native inference. Separately, the GCN’s out-of-distribution-size degradation exceeding the CNN’s ($n = 6$) is a small-sample result and should be treated as a direction for further investigation, not a settled finding about GNN generalization.
- **Combinatorial-plausibility claim is qualified, not absolute:** One exception was found in 30 live-app predictions: an input combining multiple diagrams and prose text in a single frame produced a predicted edge count exceeding the combinatorial maximum for the predicted vertex count. This appears to be an input-distribution failure specific to non-single-graph images rather than a breakdown of the property under normal use, but the claim of “never violated” should not be repeated without this caveat.
- **Pixel-only baseline interpretation:** The ink-coverage regression should be read as reassuring rather than incriminating: it explains most edge-count variance on synthetic data but almost none on real held-out data, meaning the CNN’s real-world accuracy is not attributable to simple pixel-counting. However, this comparison is itself only possible on synthetic data where the shortcut is strong; synthetic-only evaluation may still overstate how much the CNN has learned beyond low-level pixel statistics, and results on synthetic data specifically should be read with that caveat in mind.
- **Ink-fraction measurement inconsistency:** Two different scripts compute `ink_fraction` with two different background thresholds—`evaluation/image_statistics.py` (`white_threshold=250`, used for the correlation table) and `evaluation/compute_`

`structural_pixel_features.py` (`bg_threshold=200`, used for the pixel-only-baseline regression)—and were never reconciled to one canonical definition. Both give the same qualitative conclusion (ink coverage is a strong edge-count proxy on synthetic data, a weak one on real data), but the exact correlation magnitudes reported are not directly comparable to each other and should not be cited interchangeably.

- **Training reproducibility:** Two otherwise-identical full training runs (differing only in random GPU/cuDNN non-determinism on Colab) produced test overall vertex MAE of 4.5 and 3.2 respectively. Reported figures are from the later run; the gap between runs indicates the headline metrics carry some run-to-run variance not captured by the fixed random seed alone.

Read together, these limitations do not undermine the core Competitiveness finding from Section 9, but they do sharply narrow the conditions under which it holds: a graph rendered by Graphviz `sfdp`, with fewer than roughly 100 nodes, compared against a deliberately minimal-feature GCN. Extending the claim beyond that envelope—to other rendering engines, larger graphs, or richer graph-native baselines—is exactly the gap the Future Work chapter is designed to close.

11 Future Work

Each item below responds directly to one of the limitations identified in Section 10, rather than being a generic wish list; they are ordered roughly by expected effort-to-impact ratio, from cheapest to most involved.

- **Layout-augmented training:** Mix multiple layout algorithms (Kamada-Kawai, circular, hierarchical) during training to reduce layout dependence. This directly extends the layout-sensitivity probe (Section 8.3) from a single-alternative test to a trained-in robustness property, addressing the narrow scope of the current layout-robustness claim noted in the Limitations chapter.
- **Fixed node-size training:** Train models on graphs rendered with constant node sizes to force the CNN to learn true counting representation features. This is the most direct response to the confirmed node-size shortcut (Section 8.2): rather than merely detecting the confound, remove the model’s ability to rely on it at training time and see whether accuracy recovers through genuine structural learning instead.
- **Size ceiling extension:** Generate and incorporate synthetic graphs up to 500 nodes in the training pipeline to build size-extrapolation capacity. This targets the out-of-distribution size ceiling identified as the single dominant failure mode in this project (Section 8.6).
- **Stronger GNN baselines:** Evaluate graph models using Node2Vec positional encodings or spectral embeddings. This directly addresses the baseline-comparison-fairness limitation—the current GCN’s degree-only feature set was a deliberate simplification for isolating the topology-vs-geometry comparison, but a fairer head-to-head result requires re-running the comparison with a GCN given a genuinely competitive feature budget.

12 Conclusion

Returning to the two questions posed in the Introduction: on Competitiveness, the Topolens project demonstrates that deep neural networks can estimate key topological features (vertices and edges) from rendered graph drawings, achieving performance competitive with, or superior to, a basic graph-native GCN representation, across both synthetic in-distribution data and real-world held-out data. On Mechanism, the answer is more qualified: rigorous evaluation probes indicate that a significant driver of this capability is shortcut learning—specifically relying on the inverse correlation between node count and node diameter introduced by the renderer. When this cue is removed, CNN performance degrades significantly, even though the model does not collapse to pure pixel-counting either, as the ink-coverage analysis shows.

Thus, visual topology regression presents a double-edged sword: layouts provide spatial organizations that aid predictions, but networks readily overfit to arbitrary visualization standards. This has a direct practical implication for anyone considering a similar visual-inference pipeline: rendering conventions are not a neutral implementation detail to be decided late in a project, but a variable that materially shapes what the model actually learns, and should be scrutinized with the same rigor as the model architecture itself. Future research building on this project must focus on two fronts in tandem—visual standardization (or explicit augmentation against it) to reduce the model’s dependence on any single rendering convention, and OOD size-generalization, since the 100-node ceiling proved to be the single largest source of catastrophic error observed across every evaluation in this report.

13 References

The following works are cited by number, in the order they first appear in the text, wherever a specific dataset, tool, model, or statistical method is named (e.g. “NetworkX [1]”). Each citation marker links directly to its entry below.

1. Hagberg, A., Schult, D., & Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy2008)*, 11–15.
2. Erdős, P., & Rényi, A. (1959). On random graphs I. *Publicationes Mathematicae Debrecen*, 6, 290–297.
3. Barabási, A.-L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286(5439), 509–512.
4. Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of ‘small-world’ networks. *Nature*, 393(6684), 440–442.
5. Morris, C., Kriege, N. M., Bause, F., Kersting, K., Mutzel, P., & Neumann, M. (2020). TU-Dataset: A collection of benchmark datasets for learning with graphs. *ICML 2020 Workshop on Graph Representation Learning and Beyond*.

6. Debnath, A. K., Lopez de Compadre, R. L., Debnath, G., Shusterman, A. J., & Hansch, C. (1991). Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. *Journal of Medicinal Chemistry*, 34(2), 786–797.
7. Borgwardt, K. M., Ong, C. S., Schönauer, S., Vishwanathan, S. V. N., Smola, A. J., & Kriegel, H.-P. (2005). Protein function prediction via graph kernels. *Bioinformatics*, 21(suppl_1), i47–i56.
8. Gansner, E. R., & North, S. C. (2000). An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11), 1203–1233.
9. Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations (ICLR)*.
10. Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
11. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.
12. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.
13. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *IEEE International Conference on Computer Vision (ICCV)*, 618–626.
14. Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83.

Appendix

Figure & Table Index

- **Figures (in report/figures/):**
 - Figure 3.1: `graph_counts_by_tier.png` - Distribution of graph counts across the synthetic size tiers.
 - Figure 3.2: `node_edge_distributions_overall.png` - Overall node and edge count distributions across the dataset.
 - Figure 3.3: `node_edge_distributions_by_generator.png` - Node and edge distributions broken down by generator family.
 - Figure 4.1: `sample_render_grid.png` - Sample rendered images from the dataset showcasing different topologies.

- Figure 7.1a: `cnn_pred_num_vertices_scatter_test.png` - Predicted vs. true vertex count (test).
- Figure 7.1b: `cnn_pred_num_edges_scatter_test.png` - Predicted vs. true edge count (test).
- Figure 7.2a: `cnn_pred_num_vertices_scatter_held_out.png` - Predicted vs. true vertex count (held-out).
- Figure 7.2b: `cnn_pred_num_edges_scatter_held_out.png` - Predicted vs. true edge count (held-out).
- Figure 8.1: `gradcam_grid_by_generator.png` - Grad-CAM activations mapped across representatives of each generator family.
- Figure 8.2: `gradcam_grid_failure_cases.png` - Grad-CAM activations on high-error failure cases.
- Figure 8.3: `probe_variant_examples_grid.png` - Side-by-side comparison of original, constant node-size, and alternative layouts.
- Figure 8.4: `probe_variant_mae_comparison.png` - Error comparison (MAE) across the different probe variants.
- Figure 8.5a: `ink_coverage_vs_node_count.png` - Scatter plot of ink fraction vs. node count.
- Figure 8.5b: `component_size_vs_node_count.png` - Scatter plot of component area vs. node count.
- Figure 8.6a: `error_vs_num_nodes.png` - Plotting prediction error against physical graph size.
- Figure 8.6b: `error_vs_density.png` - Plotting prediction error against graph density.
- Figure 8.7: `worst_case_image_grid.png` - Grid of worst-performing outlier images yielding highest regression errors.

• **Source Tables (in `evaluation/results/`):**

- `cnn_metrics.csv`: CNN MAE/RMSE metrics by split and breakdown.
- `gnn_metrics.csv`: GNN MAE/RMSE metrics.
- `graph_statistic_metrics.csv`: Graph-statistic baseline metrics.
- `summary_comparison.csv`: Consolidated models comparison.
- `probe_summary.csv`: Probe MAE by variant and tier.
- `probe_predictions.csv`: Per-graph raw probe predictions.
- `ink_coverage_correlations.csv`: Pearson correlations coefficients.
- `failure_case_summary.csv`: MAE by failure category and split.
- `failure_case_categories.csv`: Per-graph failure category assignments.
- `baseline_predictions.csv`: Baseline per-graph predictions.
- `cnn_worst_edge_errors.csv`: Worst-error graphs for CNN edge prediction.
- `cnn_training_log.csv`: Epoch-level CNN training log.
- `gnn_training_log.csv`: Epoch-level GNN training log.
- `topolens_spotcheck_results.csv`: Spot check predictions and warnings.

- `topolens_novelimage_results.csv`: Novel-image results and plausibility.
- `topolens_free_analyses_results.csv`: Wilcoxon test results.
- `structural_pixel_features.csv`: Per-graph structural and pixel features.
- `topolens_pixel_structural_correlations.csv`: Pixel regression and collinearity stats.

Reproducibility Note

To reproduce these findings, use the following execution sequences in the project root:

1. Setup and installation:

```
python -m venv .venv
.venv/Scripts/activate
pip install -r requirements.txt
```

2. Render the synthetic and real-world graphs:

```
python data/load_tu_datasets.py
python data/generate_synthetic.py
```

3. Split the datasets and perform training:

```
python data/make_splits.py
python models/train_cnn.py
python models/gnn_baseline.py
```

4. Perform evaluations and generate results:

```
python evaluation/evaluate.py
python evaluation/ink_coverage_analysis.py
python evaluation/shortcut_probe.py
python evaluation/failure_taxonomy.py
python evaluation/compute_structural_pixel_features.py
```

All scripts use deterministic random seed initialization from `config.py` and `config.yaml` (seed = 42).